UNIT - III

STACKS, QUEUES AND LINKED LISTS

3: Make Use of Stacks and Queues to solve the given oblem.

.1 INTRODUCTION TO STACK [Oct. 22, April 23]

Stack is a linear data structure that follows LIFO (last in t out) principle, the last element inserted is the first to be pped out. It means both insertion and deletion operations ppen at one end only.

st In First Out (LIFO) Principle

Here are some real world examples of LIFO

Consider a stack of plates. When we add a plate, we add the top. When we remove, we remove from the top.

A shuttlecock box (or any other box that is closed from e end) is another great real-world example of the Last In, st Out (LIFO) principle where do insertions and removals m the same end.

epresentation of Stack Data Structure:

Stack follows Last In First Out (LIFO) Principle so the ement which is pushed last is popped first.

pes of Stack:

- **Fixed Size Stack:** As the name suggests, a fixed size stack has a fixed size and cannot grow or shrink dynamically. If the stack is full and an attempt is made to add an element to it, an overflow error occurs. If the stack is empty and an attempt is made to remove an element from it, an underflow error occurs.
- **Dynamic Size Stack:** A dynamic size stack can grow or shrink dynamically. When the stack is full, it automatically increases its size to accommodate the new element, and when the stack is empty, it decreases its size. This type of stack is implemented using a linked list, as it allows for easy resizing of the stack.

.2 STACK OPERATIONS [Oct. 22, April 23]

The array implementation of stack is not efficient from the int of view of memory utilization. The fixed size of array is uired and it remains allocated for the entire duration of the gram. Linked list implementation will have advantage over array implementation because we can allocate memory as d when it is required.

The stack using linked list consist of nodes having data and address of next node. The node definition will be,

```
class Node:
  def __init__(self, data):
    self.data = data
    self.next = None
```

A pointer called top can be declared (NODE *top) which will always point to the top of the stack. The operations push and pop will be implemented as follows:

Push Operation:

Step 1: Create a node.

Step 2: Store the data in the node.

Step 3: Link the next field of the node to the node where top is pointing.

Step 4: Point top to the recently created node.

Following figures show this operation.

Push (10):



Fig. 3.1 (a): Stack using linked list push (10)

Push (20):

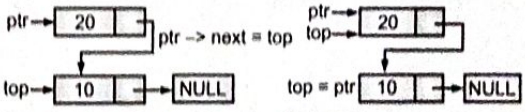


Fig. 3.1 (b): Stack using linked list push (20)

Pop Operation:

Step 1: Make a pointer temp point to the top of the stack.

Step 2: If it is NULL, then stack is empty.

Step 3: If not, remove the data from this node then advance top.

Step 4: De-allocate memory pointed by temp.

Following figures show this operation.

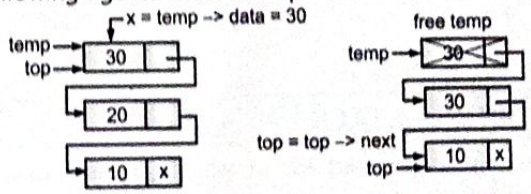


Fig. 3.2: Pop operation in stack using linked list

Following program implements stack using linked list. It has the function push and pop which has the same prototype as that used in case of array. The function push accepts data, creates a node and puts it into a linked list. The function pop removes an element from the linked list and returns it.

Program 3.1: To Implement Stack using Linked List.

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class Stack:
    def __init__(self):
        self.top = None

    def push(self, x):
        new_node = Node(x)
        new_node.next = self.top
        self.top = new_node

    def pop(self):
        if self.top is None:
            print("Stack empty")
            return -9999
        x = self.top.data
        self.top = self.top.next
        return x

# Main execution
stack = Stack()

while True:
    print("\n1. Push\n2. Pop\n3. Exit")
    ch = int(input("Enter your choice: "))

    if ch == 1:
        x = int(input("Enter a number: "))
        stack.push(x)
    elif ch == 2:
        x = stack.pop()
        if x != -9999:
            print(f"Popped value: {x}")
    elif ch == 3:
        print("Exiting...")
        break
    else:
        print("Invalid choice")
```

3.3 MULTIPLE STACK

A stack is an ordered list in which all insertions and deletions are made at one end, called the top. A queue is an ordered list in which all insertions take place at one end, the rear, while all deletions take place at the other end, the front. Given a stack S=(a[1],a[2],.....a[n]) then we say that a1 is the bottommost element and element a[i]) is on top of element a[i-1], 1<i<=n. When viewed as a queue with a[n] as the rear element one says that a[i+1] is behind a[i], 1<i<=n.

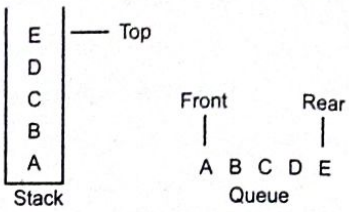


Fig. 3.3

The restrictions on a stack imply that if the elements C, D, E are added to the stack, n that order, then the element to be removed/deleted must be E. Equivalently say that the last element to be inserted into the stack will the first to be removed. For this reason, stacks are sometimes referred to as Last In First Out (LIFO) lists. The restriction queue imply that the first element which is inserted into queue will be the first one to be removed. Thus, A is the letter to be removed, and queues are known as First In Out (FIFO) lists. Note that the data object queue as defined here need not necessarily correspond to the mathematical concept of queue in which the insert/delete rules may different.

You can see the algorithms you want by clicking on items below:

- Adding an element into a stack.
- Deleting an element from a stack.
- Adding an element into a queue.
- Deleting an element from a queue.

One natural example of stacks which arises in computer programming is the processing of procedure calls and their terminations. Suppose we have four procedures as below:

The MAIN procedure invokes procedure A1. On completion of A1 execution of MAIN will resume at location r. The address r is passed to A1 which saves it in some location for later processing. A1 then invokes A2 which in turn invokes A3. In each case the invoking procedure passes the return address to the invoked procedure. If we examine the memory while A3 is computing there will be an implicit stack which looks like (q,r,s,t).

The first entry q is the address to which MAIN returns control. This list operates as a stack since the returns will be made in the reverse order of the invocations. Thus, t is removed before s, s before r and r before q. Equivalently this means that A3 must finish processing before A2, A2 before A1, and A1 before MAIN. This list of return addresses cannot be maintained in consecutive locations. For each procedure there is usually a single location associated with the machine code which is used to retain the return address. This can be severely limiting in the case of recursive and re-entrant procedures, since every time we invoke a procedure the old return address wipes out the old one. For example, if MAIN inserted a call to A1 within procedure A3 expecting the return to be at location u, then at execution time the stack would become (q, u, s, t) and the return address r would be lost. When recursion is allowed, it is no longer adequate to reserve one location for the return address of each procedure.

urns are made in the reverse order of calls, an elegant and ural solution to this procedure return problem is afforded ough the explicit use of a stack of return addresses. enever a return is made, it is to the top address in the ck.

4 APPLICATIONS OF STACK FOR EXPRESSION CONVERSION [INFIX, PREFIX AND POSTFIX] [Oct. 22, April 23]

ck can be used in number of applications such as:

- Conversion of expression.
- Evaluation of expression.
- Processing function calls.
- Handling recursive function or removal of recursion.
- Reversing a string.
- Syntax checking for Example, parenthesis check.

4.1 Expression Conversion [Oct. 22, April 23]

This is the most important application of stack. When we te a program, we write number of expressions (arithmetic, ical, etc.) in it. The compiler must interpret and convert se expressions into machine language.

An expression consists of operands and operators, For mple, the expression a + b has two operands a and b and e operator +. This is a very simple expression. Expression n be very complex consisting of number of operands and erators. While converting an expression into correct chine language format, compiler must take care of the iority of the operators. It is very difficult to directly produce code for evaluation of expression. The solution to this is, nvert the expression into a form which will not require ority of operators and the expression can be directly aluated.

An expression can be written in three different forms:

1. **Infix Expression:** Operator is in between the two operands, For Example, a + b.
2. **Postfix Expression:** Operator is after the two operands, For Example, ab+. It is called Reverse polish notation.
3. **Prefix Expression:** Operator is before the two operands, For Example, +ab. It is called Polish notation.

Suppose we have an expression in infix format a + b * c.

Its postfix is abc * + and prefix is + a * bc

If you observe postfix and prefix expression, we find that se expressions can be evaluated directly without knowing priority of operators. Whereas for infix expression we must d out priority and then evaluate.

Before we learn how to evaluate the postfix expression let see how to convert an infix expression to postfix or prefix.

If we are given an infix expression a + b * c steps for conversion will be as follows:

1. Write the expression with full parenthesized as per the priority of operators.

   For Example, (a + (b * c))
2. Replace the innermost parenthesized expression with e_1

   exp = (a + e_1) where e_1 = b * c

   Replace the next innermost parenthesized expression with e_2 as:

   i.e. exp = e_2 where e_2 = a + e_1

   Continue till you get a single expression.

   Now go in reverse direction replacing each expression with postfix or prefix

For Example, exp = ae_1 + or exp = + ae_1

exp = abc* + exp = + a * bc

Hence the postfix expression is abc * + and the prefix expression is + a * bc.

SOLVED EXAMPLE

Example 3.1: *Convert following expressions into postfix and prefix expressions.*

1. *a * b + c*
2. *a * b + c * d*
3. *a + b + c + d*
4. *a * b/c* d – e/f*
5. *a ** b * c ** d*

Solution:

1.

exp = ((a * b) + c)

= (e_1 + c) e_1 = a * b

= e_2 e_2 = e_1 + c

Postfix **Prefix**

exp = e_2 exp = e_2

= e_1c + = +e_1c

= ab * c+ = +* abc

2.

exp = ((a *b) + (c * d))

= (e_1 + e_2) e_1 = a * b e_2 = c * d

= e_2 e_2 = e_1 + e_2

Postfix **Prefix**

exp = e_2 exp = e_2

= e_1 e_2 + = +e_1 e_2

= ab * cd * + = + * ab * cd

3.

exp = (((a + b) + c) +d)

= ((e_1 + c) + d) e_1 = a + b

= (e_1 + d) e_2 = e_1 + c

= e_3 e_3 = e_2 + d

	Postfix	Prefix
	exp = e_3	exp = e_3
	= e_2d +	= +e_2d
	= e_1c + d+	= ++$e_1$cd
	= ab + c + d+	= +++ abcd

4. exp = a *b/c * d – e/f

= (((a * b)/c) * d) – (e/f)

= ((e_1/c) * d) – e_2) e_1 = a * b e_2 = e/f

= ((e_3 * d) – e_2) e_3 = e_1/c

= (e_4 – e_2) e_4 = e_3 * d

= e_5 e_5 = e_4 – e_2

	Postfix	Prefix
	exp = e_5	exp = e_5
	= e_4e_2 –	= –e_4 e_2
	= e_3d * e_2 –	= – * e_3de_2
	= e_1c/d * e_2 –	= – * /$e_1$cd e_2
	= ab * c/d * ef/ –	= – * / * abcd / ef

5. exp = a**b*c**d – e

where ** is raised to operator

= (((a**b)*(c**d)) –e)

= ((e_1 * e_2) – e) e_1 = a ** b e_2 = c ** d

= (e_3 – e) e_3 = e_1 * e_2

= e_4 e_4 = e_3 – e

	Postfix	Prefix
	exp = e_4	exp = e_4
	= e_3 e –	= –e_3 e
	= e_1 e_2 * e –	= – e * e_1 e_2
	= ab**cd***e–	= –e***ab**cd

3.5 EVALUATION OF POSTFIX EXPRESSION

[Oct. 22, April 23]

Now, let us see how we can evaluate a postfix expression. If we are given a postfix expression the expression is scanned from left till, we come across an operator. The operator corresponds to previous two operands hence operate on the operand and result is an operand for next operator. Like this go on scanning till the end of the expression.

For Example, suppose we have the expression 456 * +

The first operator is *. It will operate on 5 and 6 to give 30. The resultant expression is 430 +.

Next operator is +. It will operate on 4 and 30 to give the result is 34.

When we scan the expression and come across an operator we take latest two operands hence, we can use a stack to store the operands, so that latest two operands can be accessed. Hence, the procedure for evaluation of postfix expression will be as follows:

- Scan the expression.
- In case of operand.

  push the operand on stack.
- In case of operator

  Pop two operands from stack

  Do the operation

  Push the result on stack.
- Repeat 2 and 3 till the end of expression.

The expression can be stored in a string and can b scanned character by character. The procedure is illustrate with example below:

Suppose we have the expression 456 * +.

Expression Character	Action	Stack Content
4	push	4 …
5	push	4 5 …
6	push	4 5 6 …
*	pop 6	
	pop 5	
	push (5 * 6)	4 30 …
+	pop 30	
	pop 4	
	push (30 + 4)	34 …

Fig. 3.4: Evaluation of postfix expression 456*+

The detailed algorithm and program is:

Algorithm 3.1:

```
1. Read expression in string expr[ ]
2. i=0
3. while (expr [i] !='\0')
                    // while it is not end of string
   {
      if(expr[i] is operand)
          push (expr[i]);
      else
      {
          op1 = pop( );
          op2 = pop( );
          v = op1 (operator in expr[i]) op2;
          push (v);
      }
      i++;
   }
4. result = pop( );
5. print result
```

Program 3.2: To Evaluate Postfix Expression.

```
MAX = 10
stack = []
top = -1

def push(x):
  global top
  if top == MAX - 1:
    print("Stack is full")
  else:
    stack.append(x)
    top += 1

def pop():
  global top
  if top == -1:
    print("Stack is empty")
    return -9999
  else:
    top -= 1
    return stack.pop()

# Main Execution
expr = input("Enter postfix expression: ")

for ch in expr:
  if ch.isdigit():
    push(int(ch))
  elif ch in '+-*/':
    op2 = pop()
    op1 = pop()
    if ch == '+':
      push(op1 + op2)
    elif ch == '-':
      push(op1 - op2)
    elif ch == '*':
      push(op1 * op2)
    elif ch == '/':
      push(op1 // op2)  # integer division
  elif ch == ' ':
    continue  # skip spaces

result = pop()
print("The result is:", result)
```

3.5.1 Conversion of Infix Expression to Postfix

The evaluation of postfix expression requires stack and the priority of operators need not be considered for evaluating the expression. Normally, we write the expression in infix format. It needs to be converted into postfix format. The procedure for conversion is as follows.

A stack is used this time to store operators. The infix expression is scanned from left to right. Incase of operands, the operands are directly copied to postfix expression. In case of operators, either of two decisions is taken.

1. If priority of operator is greater than operator on top of stack, the operator is pushed on stack.
2. If priority of operator is less than operators on stack, the operators on stack are popped and copied into the output expression and the operator is pushed on to the stack.

Following examples illustrate the process of conversion of infix to postfix.

Example 1: a+b*c

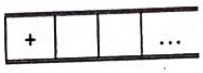
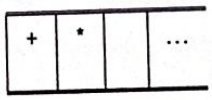
Incoming Character	Action	Postfix	Stack Contents
A	Copy to output	A	
+	Push	A	
B	Copy to output	Ab	
*	Priority more than +, Push *	Ab	
C	Copy to output	Abc	
C	Pop all operators copy to output	abc* +	

Fig. 3.5: Conversion of infix to postfix - a+b*c

Example 2: a*b+c

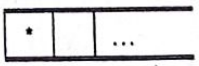
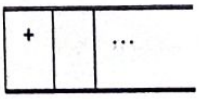
Incoming Character	Action	Postfix	Stack Contents
A	Copy to output	A	
*	Push	A	
B	Copy to output	Ab	
+	Priority less than *, pop *, copy to output, push +	ab*	
C	Copy to output	ab*c	
End of expression	Pop all operators copy to output	ab*c+	

Fig. 3.6: Conversion of infix to postfix - a*b+c

Example 3: a + b * c / d – e

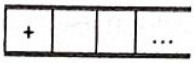
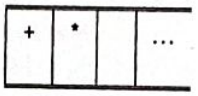
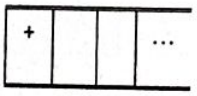
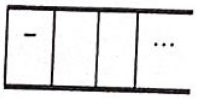
Incoming Character	Action	Postfix	Stack Contents
A	Copy to output	a	
+	Push	a	
B	Copy to output	ab	
*	Priority more than +, push	ab	
C	Copy to output	abc	
/	Priority equal to *, pop*, copy to output, push /	abc*	
D	Copy to output	abc*d	
–	Priority less than /, pop /, copy to output	abc*d/	
	priority equal to +, pop +, copy to output, push –	abd*d/+	
E	Copy to output	abc*d/+e	
End of expression	Pop all operations copy to output	abc*d/+e–	

Fig. 3.7: Conversion of infix to postfix - a + b * c / d – e

The algorithm for conversion of infix to postfix is:

Algorithm 3.2: To Convert Infix Expression to Postfix.

```
1. Read infix expression
2. i=0, j=0;
3. while (infix[i] != '\0')
   {
       if (infix[i] is operand)
       {
           postfix[j] = infix[i];
           j++;
       }
       else
       {
           if (stack empty)
               push (infix[i]);
           else
           {
if (priority (infix [i])>priority (stack[top]))
                   push (infix[i]);
               else
                   {
                       do
                       {
                           postfix[j] = pop( );
                           j++;
                       }
while (priority (infix[i])<=priority(stk[top])&& (!stack_empty);
                       push (infix[i]);
                   }
           }
       }
       i++;
   }
4. while (!stack_empty)
   {
       postfix[j] = pop( );
       j++;
   }
5. postfix[j] = '\0';
6. print postfix
7. stop.
```

For implementation of above algorithm following considerations are required:

A stack of characters needs to be defined along with the operations push and pop.

A function called priority needs to be defined which will return address of operator passed to it.

Two string variables will be required one for storing infix expression and other for postfix expression.

To make the program simple, the stack is implemented using array and it is declared globally along with the top of stack, so that we can directly access it in main function.

Only four operators +, –, *, / are taken for the conversion. Otherwise, the program will be bit complex since apart from priority; associativity of operators also plays an important role in the conversion.

The expression is assumed to have variable with single character or digit for example, a + b * c and not like sum = n1 + n2.

Program 3.3: To Convert Infix Expression to Postfix.

```
MAX = 50
stack = []
top = -1

def push(x):
  global top
  if top != MAX - 1:
    stack.append(x)
    top += 1
```

```
def pop():
  global top
  if top != -1:
    top -= 1
    return stack.pop()
  return ''

def priority(ch):
  if ch in ('*', '/'):
    return 2
  elif ch in ('+', '-'):
    return 1
  else:
    return 0

# Main execution
infix = input("Enter infix expression: ")
postfix = []
i = 0

while i < len(infix):
  ch = infix[i]
  if ch.isalpha() or ch.isdigit():
    postfix.append(ch)
  else:
    while top != -1 and priority(ch) <= priority(stack[top]):
      postfix.append(pop())
    push(ch)
  i += 1

while top != -1:
  postfix.append(pop())

print("Postfix expression:", ''.join(postfix))
```

Note: If at all you want to include the raised to operator ($ or ^) you can assign priority 3 (highest) to it.

3.6 INTRODUCTION TO QUEUE

Queue is a linear data structure in which the first element inserted is taken out first. Thus, Queue is a First In First Out (FIFO) type of a list where all insertions are made at one end called rear end and all deletions are made at the other end called front end. It is just like queue for railway reservation or buses. The first person in the queue will be first to go out.

Queue can be implemented using:

- Arrays
- Linked lists

Following four operations can be done on a queue:

1. **Insert Operation:** In this, an element is stored at a location indicated by rear.
2. **Delete Operation:** In this operation, the element at the front is removed.
3. **Queue Full:** When all the locations reserved for queue are occupied, we can't insert any more elements. This condition is queue_full condition.
4. **Queue Empty:** When there is no element stored in a queue, we can't delete any more elements. This condition is called as queue_empty.

3.7 QUEUE OPERATIONS

Queue Representation

As we now understand that in queue, we access both ends for different reasons. The following diagram given below tries to explain queue representation as data structure –

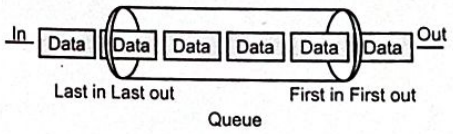


Fig. 3.8

As in stacks, a queue can also be implemented using Arrays, Linked-lists, Pointers and Structures. For the sake of simplicity, we shall implement queues using one-dimensional array.

Basic Operations

Queue operations may involve initializing or defining the queue, utilizing it, and then completely erasing it from the memory. Here we shall try to understand the basic operations associated with queues –

- **enqueue():** add (store) an item to the queue.
- **dequeue():** remove (access) an item from the queue.

Few more functions are required to make the above-mentioned queue operation efficient. These are –

- **peek():** Gets the element at the front of the queue without removing it.
- **isfull():** Checks if the queue is full.
- **isempty():** Checks if the queue is empty.

In queue, we always dequeue (or access) data, pointed by **front pointer and while enqueing (or storing) data in the queue we take help of rear pointer.**

Let's first learn about supportive functions of a queue

peek()

This function helps to see the data at the **front** of the queue. The algorithm of peek() function is as follows –

Algorithm 3.3:

```
begin procedure peek
  return queue[front]
end procedure
```

Implementation of peek() Function in Python Programming Language

Example

```
def peek():
  return queue[front]
```

isfull()

As we are using single dimension array to implement queue, we just check for the rear pointer to reach at MAXSIZE to determine that the queue is full. In case we maintain the queue in a circular linked-list, the algorithm will differ. Algorithm of isfull() function –

Algorithm 3.4:

```
begin procedure isfull

  if rear equals to MAXSIZE
    return true
  else
    return false
  endif

end procedure
```

Implementation of isfull() Function in Python Programming Language

Example

```
def is_full():
  return rear == MAXSIZE - 1
```

isempty()

Algorithm of isempty() function –

Algorithm 3.5:

```
begin procedure isempty

  if front is less than MIN  OR front is greater than rear
    return true
  else
    return false
  endif

end procedure
```

If the value of **front** is less than MIN or 0, it tells that the queue is not yet initialized, hence empty.

Here's the Python Programming code –

Example

```
def is_empty():
  return front < 0 or front > rear
```

Enqueue Operation

Queues maintain two data pointers, **front** and **rear**. Therefore, its operations are comparatively difficult to implement than that of stacks.

The following steps should be taken to enqueue (insert) data into a queue –

Step 1: Check if the queue is full.

Step 2: If the queue is full, produce overflow error and exit.

Step 3: If the queue is not full, increment **rear** pointer to point the next empty space.

Step 4: Add data element to the queue location, where the rear is pointing.

Step 5: return success.

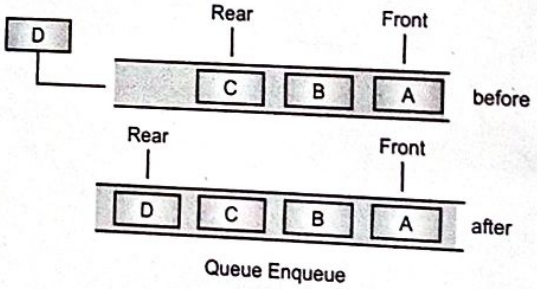


Fig. 3.9

Sometimes, we also check to see if a queue is initialized or not, to handle any unforeseen situations.

Algorithm 3.6: Algorithm for Enqueue Operation.

```
procedure enqueue(data)

  if queue is full
    return overflow
  endif

  rear ← rear + 1
  queue[rear] ← data
  return true

end procedure
```

Implementation of enqueue() in Python Programming Language

Example

```
MAXSIZE = 100
queue = [0] * MAXSIZE
rear = -1
front = 0

def is_full():
```

```
    return rear == MAXSIZE - 1

def enqueue(data):
    global rear
    if is_full():
        return 0  # Queue is full
    rear += 1
    queue[rear] = data
    return 1
```

Dequeue Operation

Accessing data from the queue is a process of two tasks – access the data where **front** is pointing and remove the data after access. The following steps are taken to perform **dequeue** operation –

Step 1: Check if the queue is empty.

Step 2: If the queue is empty, produce underflow error and exit.

Step 3: If the queue is not empty, access the data where *front* is pointing.

Step 4: Increment **front** pointer to point to the next available data element.

Step 5: Return success.

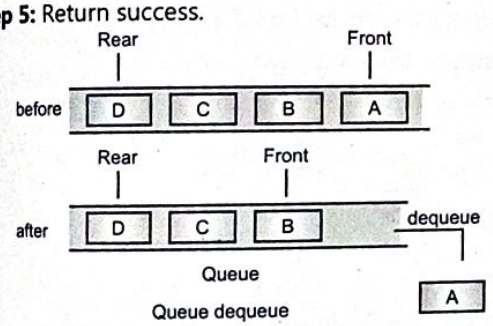


Fig. 3.10

Algorithm 3.7: Algorithm for Dequeue Operation.

```
procedure dequeue

   if queue is empty
      return underflow
   end if

   data = queue[front]
   front ← front + 1
   return true

end procedure
```

Implementation of dequeue() in Python Programming Language

Example

```
MAXSIZE = 100
queue = [0] * MAXSIZE
front = 0
rear = -1

def is_empty():
    return front < 0 or front > rear

def dequeue():
    global front
    if is_empty():
        return 0  # or raise an exception if you prefer

    data = queue[front]
    front += 1
    return data
```

3.8 CIRCULAR QUEUE AND ITS ADVANTAGES [Oct. 22, April 23]

The disadvantage of linear queue is that, even if some locations are available for storage, we will not be able to use them when rear becomes MAX −1. Hence, when rear reaches MAX −1, we should be able to come back to location 0 in case it is empty. Hence, rear ++ can be replaced by rear = (rear + 1) % MAX.

Similarly, when front reaches MAX-1 we should be able to come back to location 0 which is done using front = (front + 1) % MAX. This implementation is called circular queue. In order to handle queue full and queue empty conditions, one more variable called counter will be required. The counter c is incremented whenever insert operation is performed and decremented when delete operation is performed. The program for circular queue is as follows:

Program 3.4: To Implement Circular Queue.

```
MAX = 5
queue = [0] * MAX
front = 0
rear = -1
count = 0

def insertq(x):
    global rear, count
    if count == MAX:
        print("Queue is full")
    else:
        rear = (rear + 1) % MAX
        queue[rear] = x
        count += 1
```

```
def delq():
    global front, count
    if count == 0:
        print("Queue is empty")
        return -9999
    else:
        x = queue[front]
        front = (front + 1) % MAX
        count -= 1
        return x

# Main execution loop
while True:
    print("\n1. Insert\n2. Delete\n3. Exit")
    ch = int(input("Enter your choice: "))

    if ch == 1:
        x = int(input("Enter data: "))
        insertq(x)
    elif ch == 2:
        x = delq()
        if x != -9999:
            print(f"Deleted: {x}")
    elif ch == 3:
        print("Exiting...")
        break
    else:
        print("Invalid choice")
```

Explanation:

Let us assume queue size 5. Initially when queue is empty.

front = 0 rear = −1 c = 0

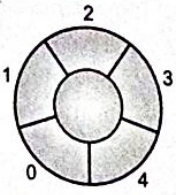


Fig. 3.11 (a): Circular queue

Insert(10) operation

front 0

rear = 0 c = 1

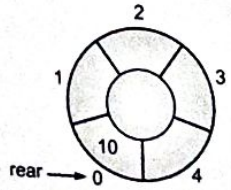


Fig. 3.11 (b): Insert (10)

Insert(20) operation

front = 0 rear = 1 c = 2

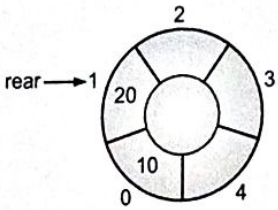


Fig. 3.11 (c): Insert (20)

Like this if we insert 30, 40, 50

front = 0 rear = 4 c = 5

The queue is full i.e., c=MAX=5

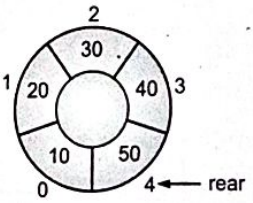


Fig. 3.11 (d): Insert (30), Insert (40), Insert (50)

Now if delete operation is performed, 10 will be removed

Before delete, front = 0 rear = 4 c = 5

After delete, front = 1 rear = 4 c = 4

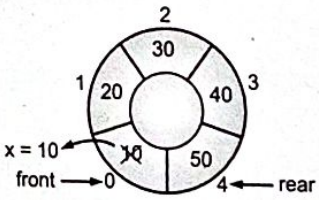


Fig. 3.11 (e): Delete operation in circular queue

One more delete operation will remove 20

Before delete; front = 1 rear = 4 c = 4

After delete, front = 2 rear = 4 c = 3

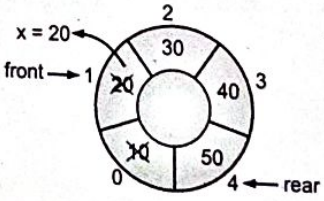


Fig. 3.11 (f): Delete operation in circular queue

Now, if we want to insert say 60,

rear = (rear + 1)% MAX.= (4 + 1)% 5 = 0 and we can store / insert the element at location 0.

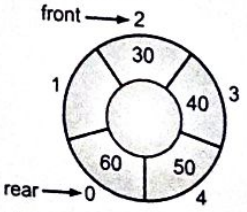


Fig. 3.11 (g): Insert operation

Thus, we can insert the element in the queue at location 0 even if rear has reached value MAX−1.

Note: If we don't use counter, the implementation becomes slightly complex, we must keep one location empty in the queue.

ircular Queue using Linked List:

A circular queue can also be implemented using circular inked list. This implementation will be most efficient than ray implementation. We can use two pointers front and rear. fact, a single pointer to last element will also be sufficient. e can access the front element with this pointer as list is ircular.

9 PRIORITY QUEUE [Oct. 22, April 23]

The priority queue is a data structure in which the dering of elements decides the two operations insert and lete. The elements from the priority queue are removed as r their priority. There are two types of priority queues:

1. Ascending order Priority Queue,
2. Descending order Priority Queue.

1. **Ascending Order Priority Queue:** It is collection of elements into which the elements are stored or inserted in any order but the smallest element can be removed.
2. **Descending Order Priority Queue:** It is collection of elements into which elements are inserted at any order but only larger element can be removed. The other operations on priority queue will be queue empty and queue full. There are several implementations of priority queues.

The priority queue can be implemented in number of ays.

ne Method can be:

- When insert operation is done, the element is stored in an array in the continuous locations.
- But when the element is removed, we must search for maximum in case of descending priority queue or minimum in case of ascending priority queue.
- In place of deleted element an invalid element (say –1) is stored.

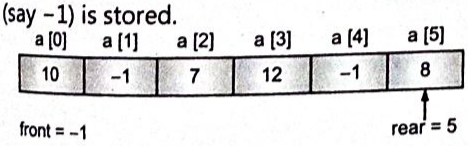


Fig. 3.12: Priority queue

When an element is inserted, we can increment rear but if ear reaches maximum size, we can do compaction i.e., earrange the array to remove invalid elements as shown in g. 3.13.

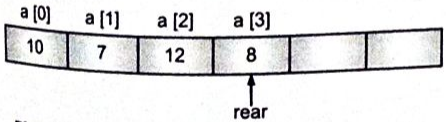


Fig. 3.13: Priority queue after compaction

econd Method can be:

Instead of maintaining the priority queue as an unordered st of elements, we can maintain it as an ordered (sorted) list f elements but then this will require more work while serting the element in the queue.

The delete operation will be simple as the minimum or maximum element is available as front end of queue.

The priority queue is used for scheduling of jobs by operating system programs. The priority queue can be used in job scheduling where the execution of job is required to be done based on the priorities. The jobs will be kept in a queue along with their priority numbers. Whenever a new job is to be executed, the job with highest priority will be taken up from the queue.

Advantages of Priority Queue

- Efficient task scheduling:
  - ➢ Tasks with higher priority are processed before lower-priority tasks.
  - ➢ Ideal for CPU scheduling, print queues, network packet management, etc.
- Dynamic priority handling:
  - ➢ Priorities can be updated or adjusted during runtime (e.g., in some implementations).
  - ➢ Useful in applications were urgency changes dynamically.
- Improved resource management:
  - ➢ Helps allocate limited resources (CPU, memory, bandwidth) to the most important jobs first.
- Efficient Implementation with Heaps:
  - ➢ Priority queues can be implemented efficiently using binary heaps.
  - ➢ Insertion: O(log n), Extraction of max/min: O(log n), Peek: O(1)
- Supports complex systems:

  Widely used in graph algorithms like:
  - ➢ Dijkstra's Algorithm
  - ➢ Prim's Minimum Spanning Tree
- Helps manage vertices based on shortest distance or minimum cost.
  - ➢ Better Than Simple Queues for Ordered Tasks: Unlike standard queues (FIFO), priority queues allow more control over the order of task execution.
  - ➢ Supports Real-Time Applications: Used in real-time systems to handle events/tasks based on urgency.
  - ➢ Multiple Priority Levels: Allows categorization and structured handling of tasks into critical, normal, and background processes.

3.10 APPLICATIONS OF QUEUE

There are several applications of queues in real world. Some of these applications are:

- Railway/Bus/Airplane Reservations.
- Processing of customer requests.
- Processing of online applications.
- In a computer system processing of jobs such as print spool (printing of multiple files/pages).

- In a operating system scheduling of jobs as per their priority in multitasking operating system.
- Categorizing of data can be useful in many problems.
- Queue simulation can be used to study performance of any application involving queue like situation.

3.10.1 Categorizing Data

We need to organize the data into groups. For example, if we have a list records having empno, name and age and we may want to group them as per their age as 21-30, 31-40, 41-50, 51-60 etc. While rearranging these records into groups, the original order is to be maintained.

The following algorithm implements categorizing of data into four groups of 1-10, 11-20, 21-30 and 31-40.

Algorithm 3.8: The Algorithm Reads Integers between 1-40 and Inserts them into Four Queues.

```
Step 1:  Read n   // Number of data items
Step 2:  i=1
Step 3:  while(i<n)
         {
              Read x   // Enter data
              if(x<=10)
                   insertq (&q1, x);
              if(x>10 && x<=20);
                   insertq(&q2, <=30);
              if(x>21, &&xx<=30);
                   insert(&q3, x);
              if(x>31 && x<=40);
                   insert(&q4, x);
              j=x+1;
         }
Step 4: Stop
```

The complete program is given below.

Program 3.5: Program for Categorizing Data.

```
Input is a list of integers between 0-39
Output is four groups (queues)*/
import random

MAX = 5

class Queue:
  def __init__(self):
    self.ar = [0] * MAX
    self.front = -1
    self.rear = -1

  def insertq(self, x):
    if self.front == MAX - 1:
      self.front = self.rear = -1
    if self.rear == MAX - 1:
      print("Queue is full")
    else:
      self.rear += 1
      self.ar[self.rear] = x

  def deleteq(self):
    if self.front == self.rear:
      # Queue empty
      return -9999
    else:
      self.front += 1
      return self.ar[self.front]

def main():
  q = [Queue() for _ in range(4)]

  n = int(input("How many numbers? "))
  i = 0
  random.seed()

  while i < n:
    print("Enter number between 1-40")
    # Original code replaces input with random

    x = random.randint(0, 99)  # Simulates rand() % 100
    print(f"Generated number: {x}")
    if 0 <= x < 40:
      q[x // 10].insertq(x)
      i += 1
    else:
      print("Invalid number")

  print("\nThe categorised data is")
  for i in range(4):
    print(f"\nData between {10 * i} to {10 * i + 9} is")
    while True:
      x = q[i].deleteq()
      if x != -9999:
        print(x, end="\t")
      else:
        break
    print()

if __name__ == "__main__":
  main()
```

3.10.2 Simulation of Queues

Any real-life application involving queue like activity can be simulated and the performance of the system can be analysed under different conditions. Thus, we can create a model of the real-life application.

Suppose a shop is open for 8 hours a day for 6 days in a week. The shop activity can be studied in different situations i.e. busy day, average day, busy hours etc. We need to simulate the activities of the shop as queue. The events that can take place are as below:

- A customer arrives
- The counter is free
- The customer is served

When the customer arrives and counter is not free, he waits in a queue. Hence, we need to create a queue of customers.

When the counter becomes free to serve next customer, we need to start a timer to find out how much time is taken by each customer. When a customer is served, we should count time taken by the server, waiting time for the customer in queue etc. This can be used to find average taken to serve each customer, average waiting time for each customer etc.

We can use random numbers to generate the inputs such as waiting time for a customer, time taken to serve a customer etc.

3.10.3 Job Scheduling

The operating system programs like Window, Unix etc., which are multitasking operating systems, execute number of programs simultaneously. It is required to do scheduling of executions of these programs. There are number of techniques for scheduling some of them are:

1. First Come First Serve (FCFS)
2. Round - Robin Technique

Both these technique uses queues for the implementation of scheduling program execution.

1. **First Come First Serve (FCFS):** The FCFS technique stores the programs in the queue on first come first serve basis. The program which comes first will be executed first. This technique is useful for scheduling of print jobs in a print server.

The disadvantages is that if one of the jobs in queue takes more time the other jobs have to wait for a long time.

2. **Round Robin Technique:** In this technique, the programs to be executed are stored in a queue.

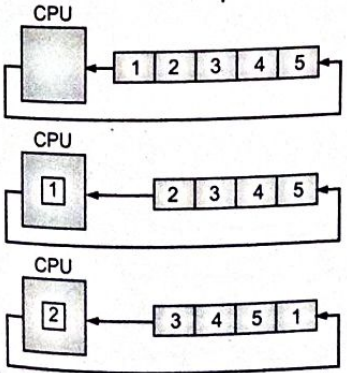


Fig. 3.14 (a): Round Robin technique

But each program is given a fix time slot for execution. If execution is over in time slot allotted, the program is removed otherwise the program is kept in the rear end of queue so that it gets another turn. The next program in the queue will be given its turn of execution for the next time slot as shown in Fig. 3.14 (a) and (b).

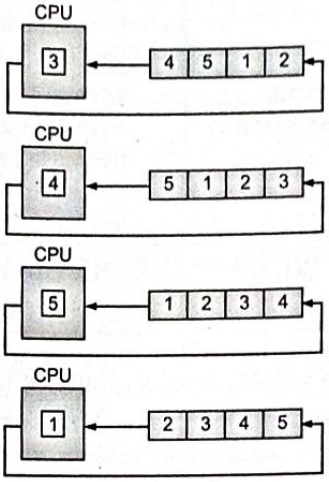


Fig. 3.14 (b): Round Robin technique

3.11 INTRODUCTION TO LINKED LISTS

A list consists of set of data items. Storage, retrieval, processing of the list is a major task required to be performed on computer. In the previous chapter, we have seen how can we store and process these lists using array. Though, it is easy to use array for storing the lists, there are two disadvantages of arrays.

We cannot increase or decrease the size of the array as per our requirement during program execution.

The insert and delete operations are computationally complex.

Another way of storing a list is using linked organisation. We can store the data items along with address or link to the next data items. It is called linked list. Since linked lists allow us to allocate memory dynamically, we can increase or decrease the size of the list as per the number of elements to be stored. The linked lists make it easy to insert or delete the elements.

We summarize below the difference between arrays and linked lists.

Arrays	Linked Lists
1. Size of array needs to be mentioned while writing program. Hence fixed memory location is assigned for storage of data items.	1. Memory is allocated as and when it is required for storing data items in the linked list.
2. Data items are stored in contiguous (adjoining) memory locations.	2. Data items not in contiguous memory locations.
3. No overhead of storing address of another node.	3. Overhead of storing address of another node/structure.
4. Element in the array is difficult to insert or delete.	4. It becomes very easy to insert or delete.
5. Random access of any data	5. To access an element in

Arrays	Linked Lists
element in an array is possible.	linked list we have to traverse the list from beginning.
6. When an items is not required its space cannot be utilised by another variable.	6. When an item is no longer needed its space can be returned to the system which can be used by another variable.

3.12 PRIMITIVE OPERATIONS ON LINKED LIST [Oct. 22, April 23]

A linked list is a linear data structure where elements (nodes) are stored in non-contiguous memory locations. Each node contains two parts: the data and a pointer to the next node. Unlike arrays, linked lists provide dynamic memory allocation, making insertion and deletion operations more efficient in certain scenarios.

Below are the core primitive operations performed on a singly linked list:

1. Create

Creating a linked list involves defining the node structure and dynamically allocating memory for each node using the new operator. Each new node is linked to the list by assigning the appropriate next pointer.

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

def create_node(value):
    return Node(value)
```

2. Traverse

Traversal refers to visiting each node in the list starting from the head and processing its data (e.g., printing).

```
def traverse(head):
    temp = head
    while temp is not None:
        print(temp.data, end=" -> ")
        temp = temp.next
    print("None")
```

3. Search

Searching means checking whether a particular element exists in the list. The search is typically performed in a linear fashion.

```
def search(head, key):
    temp = head
    while temp is not None:
        if temp.data == key:
            return True
        temp = temp.next
    return False
```

4. Insert

There are multiple ways to insert a node: at the beginning, at the end, or after a specific node.

Insert at Beginning:

```
def insert_beginning(head, value):
    new_node = create_node(value)
    new_node.next = head
    return new_node  # new head
```

Insert at End:

```
def insert_end(head, value):
    new_node = create_node(value)
    if head is None:
        return new_node
    temp = head
    while temp.next is not None:
        temp = temp.next
    temp.next = new_node
    return head
```

Insert After a Specific Node:

```
def insert_after(head, after_value, new_value):
    temp = head
    while temp is not None and temp.data != after_value:
        temp = temp.next
    if temp is None:
        return  # after_value not found
    new_node = create_node(new_value)
    new_node.next = temp.next
    temp.next = new_node
```

5. Delete

Deleting nodes can also be done from the beginning, the end, or by value.

Delete from Beginning:

```
def delete_beginning(head):
    if head is None:
        return None
    return head.next
```

Delete from End:

```
def delete_end(head):
    if head is None:
        return None
    if head.next is None:
        return None
    temp = head
    while temp.next.next is not None:
        temp = temp.next
    temp.next = None
    return head
```

Delete by Value:

```
def delete_by_value(head, key):
    if head is None:
        return None
    if head.data == key:
        return head.next
    temp = head
    while temp.next is not None and temp.next.data != key:
        temp = temp.next
    if temp.next is None:
        return head  # key not found
    temp.next = temp.next.next
    return head
```

Sort

Sorting a linked list can be done using basic algorithms like bubble sort. The node data is swapped instead of rearranging node pointers for simplicity.

```
def sort_list(head):
    if head is None:
        return
    i = head
    while i.next is not None:
        j = i.next
        while j is not None:
            if i.data > j.data:
                i.data, j.data = j.data, i.data
            j = j.next
        i = i.next
```

Concatenate

Concatenation joins two lists into a single one by linking the last node of the first list to the head of the second list.

```
def concatenate(head1, head2):
    if head1 is None:
        return head2
    temp = head1
    while temp.next is not None:
        temp = temp.next
    temp.next = head2
    return head1
```

3.13 TYPES OF LINK LIST [Oct. 22, April 23]

3.13.1 Singly/ Linear Linked List

Linked list is a set of structures (called nodes) having one of the fields as pointer to similar structure. (nodes)

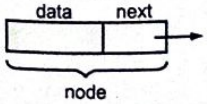


Fig. 3.15: Node in a linked list

The data field can be integers/real/string/record and next field is a pointer containing address of similar node.

A linked list is represented as:

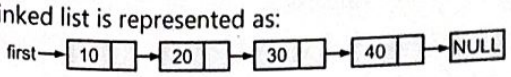


Fig. 3.16: Singly linked list

Here, first is a pointer containing address of first node of the linked list.

Also note that the last node has next field pointing to NULL means it is referring nowhere.

Before we go into details of linked list, we discuss some basic concepts.

1. **Static Variable:** These variables are declared and named while writing program. For example,

```
a,b;
```

the space allotted to these variables exists as long as program is running.

2. **Dynamic Variables:** They are created during program execution as and when required. These variables are accessed using pointers. They can be destroyed when they are not required.
3. **Defining a Node in Linked List:** Suppose that we have to store a list of integer numbers in a linked list following structure variable can be defined.

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```

Here we are declaring a structure called as node. It has two fields data and next; data to store integer number and next to store address of another similar structure. (This is called as self referential structure),

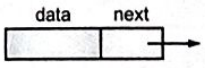


Fig. 3.17: Self referential structure

```
ptr = None  # Initially, ptr points to nothing
```

In Python, a variable like ptr can be used to hold a reference to an object of a custom class, such as PersonNode. This is similar to how a pointer in C (struct node *ptr) stores the address of a structure variable.

If we want to store a list of persons with attributes like **name** and **age**, we can define a node class as follows:

```
class Node:
    def __init__(self, name, age):
        self.name = name      # string: person's name
        self.age = age        # int: person's age
        self.next = None      # reference to the next node
```

4. How to Allocate and Deallocate Memory?

To allocate memory to a dynamic variable we use malloc function as

```
ptr = Node("Name", 25) # or default: Node("", 0)
```

when we do this a block of memory of the size of struct node is allocated to ptr. i.e. ptr starts pointing to the node as shown in Fig. 3.18.



Fig. 3.18: Pointer to a node

Thus, ptr has address of the node.

If we have to assign the data to this structure pointed by ptr, we can write

```
ptr.data = x
```

where, x is an integer number.

To de-allocate the memory allocated to ptr we can write

```
ptr = None  # Removes reference to the object
```

The memory allocated to ptr will be returned back to system so that system can reuse it.

Note: When malloc fails to allocate memory, it returns NULL.

5. Some Linked List Related Statements: Let us consider that we have pointers,

```
ptr1 = ptr2 = ptr3 = None  # initially, they point to nothing
```

The following statements will result into the 3 pointers pointing to 3 locations (Structures) as shown in Fig. 3.19.

```
# Create three Node references
ptr1 = Node(10)
ptr2 = Node(20)
ptr3 = Node(30)
```

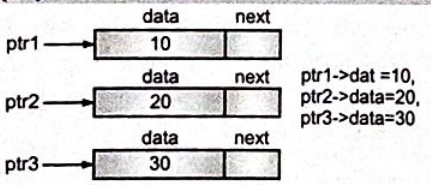


Fig. 3.19: Memory allocation and data storage

Now if we write

(i) ptr1->next=ptr2

We are storing address of second node (pointed by ptr2) in the next field of first node (pointed by ptr1). You can read this as ptr1's next will start pointing to a place where ptr2 is pointing.

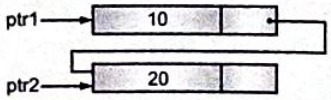


Fig. 3.20: Linking two nodes

(ii) temp=ptr1;

temp will no longer point to current location. It will start pointing to a node pointed by ptr1. You can read this as temp will start pointing to a place where ptr1 is pointing

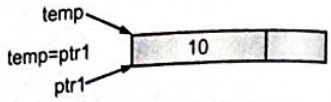


Fig. 3.21: Another pointer to a node

(iii) The following statements will result in creation of a l list.

(a) ptr1->next=ptr2;

(b) ptr2->next=ptr3;

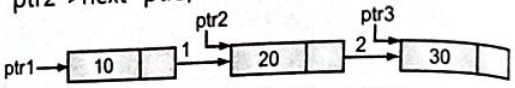


Fig. 3.22: Creating linked list

(iv) If we write temp=ptr1->next, temp will start pointing place where ptr1-> next is pointing.

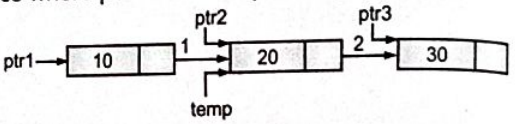


Fig. 3.23: Pointer to a next node

(v) ptr1->next=ptr->3

It will make ptr1->next point to the node pointe ptr2->next. It will stop pointing to the node pointed by pt

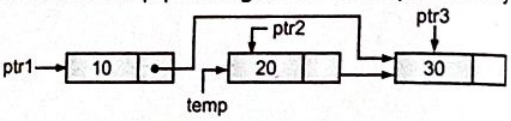


Fig. 3.24: Eliminating a node

Now let us write a simple program to create a linke and display the elements in it.



Fig. 3.25: Singly linked list

Program 3.6: A linked List of Three Elements.

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

# Create nodes
ptr1 = Node(10)
ptr2 = Node(20)
ptr3 = Node(30)

# Link nodes
ptr1.next = ptr2
ptr2.next = ptr3
ptr3.next = None  # optional, default is already None

# Traverse and print
temp = ptr1
while temp is not None:
    print(temp.data)
    temp = temp.next
```

Explanation:

1. In the first seven statements, memory is allocated to ptr1, ptr2, ptr3 and data is stored in the data fields.

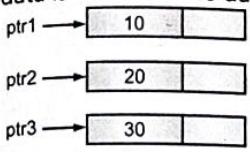


Fig. 3.26: Memory allocation and storage in nodes

2. ptr1->next=ptr2
   ptr2->next=ptr3
   ptr3->next=NULL;

will create a linked list and the various pointers will be pointing as below.

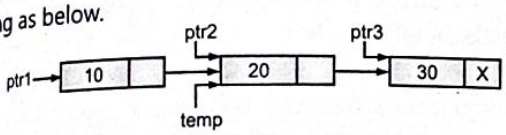


Fig. 3.27: Linking nodes

3. Next, to display the elements in the list we point temp to ptr1 and go on advancing the pointer while displaying the data in each node.

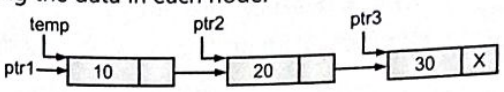


Fig. 3.28: Pointer to a first node

Above program is very elementary. Now let us write a program to create a linked list of n elements (i.e. generalized linked list). To create a list of integer numbers, we first read number of elements in the list. Then we go on allocating memory for each node and establishing link between them. Following is the program which creates and displays the linked list.

Program 3.7: To Create and Display a Linked List.

```
class Node:
  def __init__(self, data):
    self.data = data
    self.next = None

first = None
temp = None

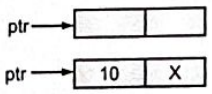
n = int(input("Enter how many elements: "))

for i in range(n):
  x = int(input("Enter data: "))
  ptr = Node(x)

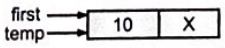
  if first is None:
    first = ptr
    temp = ptr
  else:
    temp.next = ptr
    temp = ptr

# Traverse and print the linked list
temp = first
while temp is not None:
  print(f"{temp.data}->", end='')
  temp = temp.next

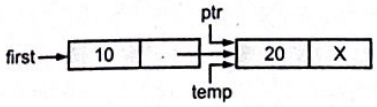
print("NULL")
```



/*if it is first node */



/*if it is not first node*/



Explanation:

1. We read number of elements in the list in variable n.
2. We go on reading data in x. Allocate memory for ptr, store the data in x in the data field of ptr. If we are storing first element the condition first==NULL will be true. We should point the pointer first to this node (if part). Otherwise, we link previous node with current and advance the pointer temp (else part).
3. To display the elements in the linked list we first point the pointer temp to first node. Access data and advance temp

to next node. We continue this till temp doesn't reference to NULL.

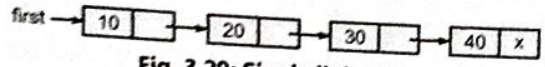


Fig. 3.29: Singly linked list

Now let us write separate functions for create and display the linked list:

Program 3.8: To Create Linked Lists.

```
#include <stdio.h>
#include <stdlib.h>

typedef struct node {
   int data;
   struct node *next;
} NODE;

NODE* create() {
   NODE *first = NULL, *ptr, *temp;
   int i, x, n;

   printf("Enter number of elements:\n");
   scanf("%d", &n);

   for(i = 1; i <= n; i++) {
      printf("Enter data:\n");
      scanf("%d", &x);
      ptr = (NODE *)malloc(sizeof(NODE));
      ptr->data = x;
      ptr->next = NULL;

      if(first == NULL)
         first = ptr;
      else
         temp->next = ptr;

      temp = ptr;
   }
   return first;
}

void disp(NODE *temp) {
   while(temp != NULL) {
      printf("%d->", temp->data);
      temp = temp->next;
   }
   printf("NULL\n");
}

int main() {
   NODE *l1, *l2;
   l1 = create();
   l2 = create();
   disp(l1);
   disp(l2);
   return 0;
}
```

Explanation:

1. The create function when called from main function creates a linked list which is pointed by first pointer. The address in linked list in pointer first is return back in the main function to a pointer variable say *l1* or *l2*. The pointer variable *l1* and *l2* in a main function now as address of linked list created by the create function.

2. The function disp displays the elements in the linked list, it required the address of linked list to be passed to it. The pointer variable temp collects this address and traverses the list sequentially.

3.13.2 Implementation of Singly /Linear Linked List

Now we can write a complete menu driven program to implement operations viz. create, display, search, insert, delete in a linked list. For this we have to write separate function for these operations.

Program 3.9: Implementation of Singly Linked List with Operations Create, Display, Insert, Delete and Search.

```
class Node:
   def __init__(self, data):
      self.data = data
      self.next = None

def create():
   n = int(input("Enter number of elements: "))
   first = None
   temp = None
   for _ in range(n):
      x = int(input("Enter data: "))
      ptr = Node(x)
      if first is None:
         first = ptr
      else:
         temp.next = ptr
      temp = ptr
   return first

def disp(head):
   temp = head
   while temp is not None:
      print(f"{temp.data}->", end="")
      temp = temp.next
   print("NULL")
```

```
def search(head, x):
    temp = head
    pos = 1
    while temp is not None:
        if temp.data == x:
            return pos
        temp = temp.next
        pos += 1
    return -1

def insert(head, x, p):
    new_node = Node(x)
    if p == 1:
        new_node.next = head
        return new_node
    temp = head
    i = 1
    while i < p - 1 and temp is not None:
        temp = temp.next
        i += 1
    if temp is None:
        print("Invalid position")
        return head
    new_node.next = temp.next
    temp.next = new_node
    return head

def delete(head, p):
    if head is None:
        print("List is empty")
        return head
    if p == 1:
        temp = head
        head = head.next
        del temp
        return head
    temp = head
    i = 1
    prev = None
    while i < p and temp is not None:
        prev = temp
        temp = temp.next
        i += 1
    if temp is None:
        print("Invalid position")
        return head
    prev.next = temp.next
    del temp
    return head

def main():
    head = None
    while True:
        print("\n1. Create\n2. Display\n3. Search\n4. Insert\n5. Delete\n6. Exit")
        ch = int(input("Enter your choice: "))
        if ch == 1:
            head = create()
        elif ch == 2:
            if head is None:
                print("List is empty")
            else:
                disp(head)
        elif ch == 3:
            x = int(input("Enter number to be searched: "))
            pos = search(head, x)
            if pos == -1:
                print("Not found")
            else:
                print(f"Found at position {pos}")
        elif ch == 4:
            x = int(input("Enter data to insert: "))
            p = int(input("Enter position to insert: "))
            head = insert(head, x, p)
        elif ch == 5:
            p = int(input("Enter position to delete: "))
            head = delete(head, p)
        elif ch == 6:
            print("Exiting...")
            break
        else:
            print("Invalid choice")

if __name__ == "__main__":
    main()
```

3.13.3 Circular Linked List (CLL)

In singly linked list we can access the data only in forward direction. Given a pointer to a node in a singly linked list, we can access all the node which follow that node. But we can not access the previous node.

In circular linked list, we store the first node's address in the next field of last node, so that the list becomes circular. In this case, given a pointer to any node we can access all the nodes in the list.

Truly speaking circular linked list does not have first or last node. Any node can be treated as a first node and the node before it can be last. But as a convention and convenience we can have pointer to last node and the next will be first node.

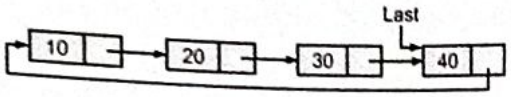


Fig. 3.30: Circular linked list

Some of the algorithms where we need to frequently move in the forward or backward direction are easily implemented using circular linked list.

There can be different ways of implementation of CLL.

Following program implements a circular linked list. The circular linked list is created in such a way that at every stage, the list will be circular, i.e., right from insertion of first node, the list will be circular. The rest of the operations insert, delete, search etc. will be similar to singly linked list.

Program 3.10: To Implement Various Operations on DLL.

```
class Node:
  def __init__(self, data):
    self.data = data
    self.next = None

def create():
  n = int(input("Enter number of elements\n"))
  last = None
  for _ in range(n):
    x = int(input("Enter data\n"))
    ptr = Node(x)
    if last is None:
      last = ptr
      last.next = last  # Circular link for single node
    else:
      ptr.next = last.next
      last.next = ptr
      last = ptr
  return last

def disp(last):
  if last is None:
    print("List is empty")
    return
  temp = last.next
  while temp != last:
    print(f"{temp.data}->", end="")
    temp = temp.next
  print(last.data)

def main():
  last = create()
  disp(last)

if __name__ == "__main__":
  main()
```

Explanation:

1. The pointer 'last' points to the last node in the list
2. To create a list we start with last=NULL.
   Data is accepted stored in a node ptr.
   If it is first element last will point to it.

   Second element onward the node is linked to last and next field is linked to first node which is nothing last->next and the new node is made last.
3. To display, we start with first node (i.e. last->ne and advance till we reach the last node.

3.13.4 Doubly Circular Linked List

Circular doubly linked list is a more complex type of structure in which a node contains pointers to its prev node as well as the next node. Circular doubly linked doesn't contain NULL in any of the node. The last node of list contains the address of the first node of the list. The node of the list also contains address of the last node in previous pointer.

A circular doubly linked list is shown in the follow Fig. 3.31.

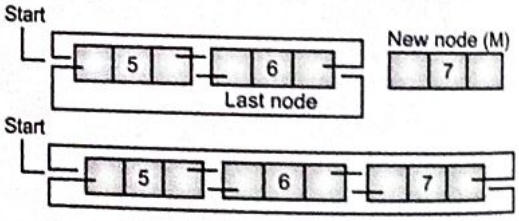


Fig. 3.31

Due to the fact that a circular doubly linked list conta three parts in its structure therefore, it demands more sp per node and more expensive basic operations. However circular doubly linked list provides easy manipulation of pointers and the searching becomes twice as efficient.

Memory Management of Circular Doubly Linked List

The Fig. 3.32 shows the way in which the memory allocated for a circular doubly linked list. The variable he contains the address of the first element of the list i.e. 1 her the starting node of the list contains data A is stored address 1. Since, each node of the list is supposed to ha three parts therefore, the starting node of the list contai address of the last node i.e. 8 and the next node i.e. 4. The la node of the list that is stored at address 8 and containing da as 6, contains address of the first node of the list as shown the image i.e. 1. In circular doubly linked list, the last node identified by the address of the first node which is stored the next part of the last node therefore the node whi contains the address of the first node, is actually the last no of the list.

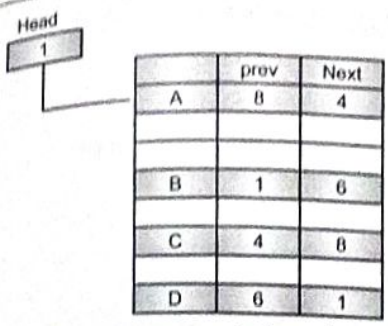


Fig. 3.32: Memory representation of a circular doubly limked list

Operations on Circular Doubly Linked List:

There are various operations which can be performed on circular doubly linked list. The node structure of a circular doubly linked list is similar to doubly linked list. However, the operations on circular doubly linked list is described in the following table.

S. N.	Operation	Description
1.	Insertion at beginning	Adding a node in circular doubly linked list at the beginning.
2.	Insertion at end	Adding a node in circular doubly linked list at the end.
3.	Deletion at beginning	Removing a node in circular doubly linked list from beginning.
4.	Deletion at end	Removing a node in circular doubly linked list at the end.

Traversing and searching in circular doubly linked list is similar to that in the circular singly linked list.

Program 3.11: Program to Implement All the Operations on Circular Doubly Linked List.

```
class Node:
  def __init__(self, data):
    self.data = data
    self.prev = None
    self.next = None

class CircularDoublyLinkedList:
  def __init__(self):
    self.head = None

  def insertion_beginning(self, item):
    new_node = Node(item)
    if self.head is None:
      new_node.next = new_node.prev = new_node
      self.head = new_node
    else:
      last = self.head.prev

      new_node.next = self.head
      new_node.prev = last
      last.next = self.head.prev = new_node
      self.head = new_node
    print("\nNode inserted at beginning")

  def insertion_last(self, item):
    new_node = Node(item)
    if self.head is None:
      new_node.next = new_node.prev = new_node
      self.head = new_node
    else:
      last = self.head.prev

      last.next = new_node
      new_node.prev = last
      new_node.next = self.head
      self.head.prev = new_node
    print("\nNode inserted at end")

  def deletion_beginning(self):
    if self.head is None:
      print("\nUNDERFLOW - List is empty")
      return
    if self.head.next == self.head:
      self.head = None
      print("\nNode deleted from beginning")
      return
    last = self.head.prev
    self.head = self.head.next
    last.next = self.head
    self.head.prev = last
    print("\nNode deleted from beginning")

  def deletion_last(self):
    if self.head is None:
      print("\nUNDERFLOW - List is empty")
      return
    if self.head.next == self.head:
      self.head = None
      print("\nNode deleted from end")
      return
    last = self.head.prev
    second_last = last.prev
    second_last.next = self.head
    self.head.prev = second_last
    print("\nNode deleted from end")

  def display(self):
    if self.head is None:
```

```
            print("\nNothing to print - list is empty")
            return
        print("\nPrinting values:")
        temp = self.head
        while True:
            print(temp.data)
            temp = temp.next
            if temp == self.head:
                break

    def search(self, item):
        if self.head is None:
            print("\nEmpty list")
            return
        temp = self.head
        pos = 1
        found = False
        while True:
            if temp.data == item:
                print(f"Item found at location {pos}")
                found = True
                break
            temp = temp.next
            pos += 1
            if temp == self.head:
                break
        if not found:
            print("Item not found")

def main():
    cdll = CircularDoublyLinkedList()

    while True:
        print("\n*********Main Menu*********")
        print("1. Insert at Beginning")
        print("2. Insert at End")
        print("3. Delete from Beginning")
        print("4. Delete from End")
        print("5. Search")
        print("6. Show")
        print("7. Exit")

        choice = int(input("Enter your choice: "))

        if choice == 1:
            val = int(input("Enter item value: "))
            cdll.insertion_beginning(val)
        elif choice == 2:
            val = int(input("Enter item value: "))
            cdll.insertion_last(val)
        elif choice == 3:
            cdll.deletion_beginning()
        elif choice == 4:
            cdll.deletion_last()
        elif choice == 5:
            val = int(input("Enter item to search: "))
            cdll.search(val)
        elif choice == 6:
            cdll.display()
        elif choice == 7:
            print("Exiting...")
            break
        else:
            print("Please enter valid choice.")

if __name__ == "__main__":
    main()
```

Output:

```
*********Main Menu*********
Choose one option from the following list ...
==========================================
1. Insert in Beginning
2. Insert at last
3. Delete from Beginning
4. Delete from last
5. Search
6. Show
7. Exit
Enter your choice?
1
Enter Item value123
Node inserted
*********Main Menu*********
Choose one option from the following list ...
==========================================
1. Insert in Beginning
2. Insert at last
3. Delete from Beginning
4. Delete from last
5. Search
6. Show
7. Exit
Enter your choice?
2
Enter value234
node inserted
```

```
*********Main Menu*********
Choose one option from the following list ...
==================================
1. Insert in Beginning
2. Insert at last
3. Delete from Beginning
4. Delete from last
5. Search
6. Show
7. Exit
Enter your choice?

Enter Item value90
Node inserted
*********Main Menu*********
Choose one option from the following list ...
==================================
1. Insert in Beginning
2. Insert at last
3. Delete from Beginning
4. Delete from last
5. Search
6. Show
7. Exit
Enter your choice?
2
Enter value80
Node inserted
*********Main Menu*********
Choose one option from the following list ...
==================================
1. Insert in Beginning
2. Insert at last
3. Delete from Beginning
4. Delete from last
5. Search
6. Show
7. Exit
Enter your choice?
3
*********Main Menu*********
Choose one option from the following list ...
==================================
1. Insert in Beginning
2. Insert at last
3. Delete from Beginning
4. Delete from last
5. Search
6. Show
7. Exit
Enter your choice?
4
node deleted
*********Main Menu*********
Choose one option from the following list ...
==================================
1. Insert in Beginning
2. Insert at last
3. Delete from Beginning
4. Delete from last
5. Search
6. Show
7. Exit
Enter your choice?
6
printing values ...
123
*********Main Menu*********
Choose one option from the following list ...
==================================
1. Insert in Beginning
2. Insert at last
3. Delete from Beginning
4. Delete from last
5. Search
6. Show
7. Exit
Enter your choice?
5
Enter item which you want to search?
123
item found at location 1
*********Main Menu*********
Choose one option from the following list ...
==================================
1. Insert in Beginning
2. Insert at last
3. Delete from Beginning
4. Delete from last
5. Search
6. Show
7. Exit
Enter your choice?
7
```

3.13.5 Doubly Linked List

A doubly linked list is a set of linked nodes in which each node stores data as well as address of next and previous nodes. In singly linked list we can move only in forward direction. Doubly linked list allows us to move in both directions. Hence, given pointer to a node we can access any element which is either after the node or before the node. Some of the algorithm can be implemented with better time complexity using doubly linked list. But this list requires additional space in each node for storing previous node address. The implementation of the list is complex compared to singly linked list.

A node in the doubly linked list can be defined as below:

```
class Node:
    def __init__(self, data):
        self.data = data      # Value of the node
        self.next = None      # Pointer to the next node
        self.prev = None      # Pointer to the previous node
```



Fig. 3.33: NODE in a doubly linked list

The field data stores data element. The field next stores address of the next node and the field prey stores address of the previous node. A doubly linked list of integers is shown in Fig. 3.34.

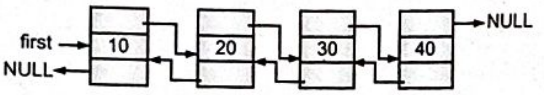


Fig. 3.34: Doubly linked list

Now let us discuss various operations on doubly linked list.

The node definition will be as follows:

```
class Node:
    def __init__(self, data):
        self.data = data      # Node's data
        self.next = None      # Reference to next node
        self.prev = None      # Reference to previous node
```

1. Create

```
class Node:d
    def __init__(self, data):
        self.data = data
        self.next = None
        self.prev = None

def create():
    first = None
    temp = None

    n = int(input("How many elements? "))
    for _ in range(n):
        x = int(input("Enter data: "))
        ptr = Node(x)  # Create new node

        if first is None:
            first = ptr  # First node
        else:
            temp.next = ptr  # Forward link
            ptr.prev = temp  # Backward link

        temp = ptr  # Advance temp

    return first
```

The process is shown in Fig. 3.35.

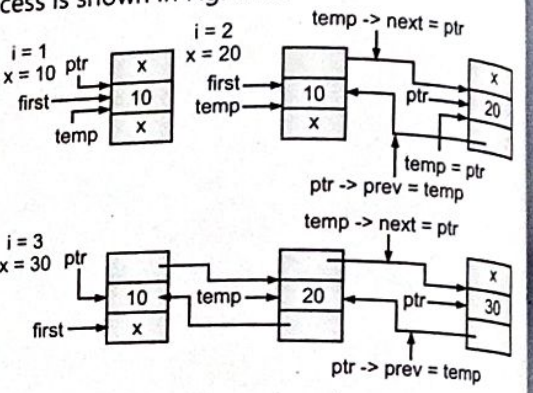


Fig. 3.35: Create procedure of DLL

2. Display

```
def disp(first):
    temp = first
    print("DLL in forward direction")
    # Traverse forward
    while temp.next is not None:
        print(f"{temp.data}->", end="")
        temp = temp.next
    print(temp.data)  # Print last node data

    print("DLL in reverse direction")
    # Traverse backward
    while temp is not None:
        print(f"{temp.data}<-", end="")
        temp = temp.prev
    print()  # New line after reverse print
```

3.13.6 Insert Operation in Doubly Linked List

To insert an element, we get a pointer (say temp) to node after which new element to be inserted. Then the node (say ptr) links to temp in backward direction and next node after temp in forward direction. It is shown Fig. 3.36, where 30 is inserted in between 20 and 40.

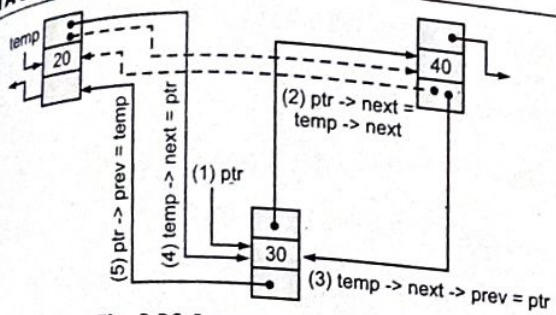


Fig. 3.36: Insert operation in DLL

It means during insert operation in DLL, four new links are created and two links are deleted. If we are inserting before first node, only two new links are established.

Algorithm 3.9: To Insert a Node in DLL.

```
Read p position where new node is to be inserted.
i=1;
temp= first;
while (i<p&&temp!= NULL)
{
    temp=temp->next;
    i++;
}
if (temp!=NULL)
{
    Read x;
    ptr=malloc (sizeof (NODE))
    ptr->data=x;
    ptr->next=ptr->prev=NULL;
    if (p==0) // Insertion before first node
    {
        ptr->next=temp;
        temp->prev=ptr;
        first=ptr;
    }
    else      // Insertion after first node
    {
        ptr->next=temp->next;
        temp->next->prev=ptr;
        temp->next=ptr;
        ptr->prev temp;
    }
}
else
    printf("Invalid position");
stop.
```

3.13.7 Delete Operation in Doubly Linked List

To delete node in DLL we get a pointer (say temp) to a node to be deleted. With the help of this pointer a node previous to temp and next to temp are linked to each other and temp is eliminated from the list. The operation is shown in Fig. 3.37.

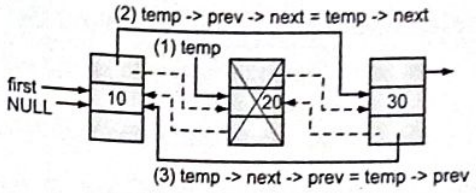


Fig. 3.37: Delete operation

It means during delete operation in DLL, two new links are established and four links are removed. If we are deleting first node, only two links are deleted.

Algorithm 3.10: To Delete a Node in DLL.

```
1. Read p
2. i=1.
3. temp=first
4. while (i<p&&temp!= NULL)
   {
       temp=temp->next;
       i++;
5. if (temp!=NULL)
   {
       if (p==1)          // First node
       {
           first=first->next;
           first->prev=NULL;
           free (temp);
       }
       else      // Any other node after first
       {
           temp->prev->next=temp->next;
           temp ->next->prev=temp->prev;
           free (temp);
       }
   }
   else
       printf("Invalid position");
6. stop.
```

3.13.8 Search Operation in Doubly Linked List

To search an element, we start from the first node and keep on advancing in the list till we get the data at a particular node.

Algorithm 3.11: To Search Data in DLL.

```
1. Read s (data to be searched)
2. temp=first;
3. while (temp->data!=s&&temp!=NULL)
       temp=temp->next;
4. if (temp!=NULL)
       printf("Found");
   else
       printf("Not found");
```

3.13.9 Implementation of Doubly Linked List

Now let us write a complete program to implement a DLL. The functions used in the program are:

1. **Create:** Creates a DLL and returns the address of first node.

```
node *create_dll( );
```

2. **Display:** Displays the list in forward and reverse direction.

```
void disp_dll(NODE*);
```

3. **Insert:** Inserts a new node in DLL and returns the address of first node.

```
NODE *insert_dll(NODE*, int, int);
```

4. **Delete:** Deletes a node in DLL and returns the address of first node.

```
NODE *del_dll(NODE*, int);
```

5. **Search:** Finds a node with given data and returns position of the node.

```
int *search_dll(NODE*, int);
```

Program 3.12: To Implement Various Operations on DLL.

```
class Node:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None
def create_dll():
    n = int(input("How many elements? "))
    first = None
    temp = None
    for _ in range(n):
        x = int(input("Enter data: "))
        ptr = Node(x)
        if first is None:
            first = ptr
        else:
            temp.next = ptr
            ptr.prev = temp
        temp = ptr
    return first
def disp_dll(first):
    if first is None:
        print("List is empty")
        return
    print("DLL in forward direction")
    temp = first
    while temp.next is not None:
        print(f"{temp.data}->", end="")
        temp = temp.next
    print(temp.data)
    print("DLL in reverse direction")
    while temp is not None:
        print(f"{temp.data}<-", end="")
        temp = temp.prev
    print()
def search_dll(first, x):
    temp = first
    pos = 1
    while temp is not None:
        if temp.data == x:
            return pos
        temp = temp.next
        pos += 1
    return -1
def insert_dll(first, x, p):
    ptr = Node(x)
    if p == 1:  # Insert at beginning
        ptr.next = first
        if first:
            first.prev = ptr
        first = ptr
        return first
    temp = first
    i = 1
    while i < p - 1 and temp is not None:
        temp = temp.next
        i += 1
    if temp is None:
        print("Invalid position")
        return first
    ptr.next = temp.next
    if temp.next:
        temp.next.prev = ptr
    temp.next = ptr
    ptr.prev = temp
    return first
def delete_dll(first, p):
    if first is None:
        print("List is empty")
        return first
    temp = first
    if p == 1:
        first = first.next
        if first:
            first.prev = None
        return first
    i = 1
    while i < p and temp is not None:
        temp = temp.next
        i += 1
    if temp is None:
        print("Invalid position")
        return first
    if temp.next:
        temp.next.prev = temp.prev
    if temp.prev:
        temp.prev.next = temp.next
    return first
def main():
    first = None
    while True:
        print("\n1. Create\n2. Display\n3. Search\n4. Insert\n5. Delete\n6. Exit")
        ch = int(input("Enter your choice: "))
        if ch == 1:
```

```
    first = create_dll()
  elif ch == 2:
    disp_dll(first)
  elif ch == 3:
    x = int(input("Enter number to be searched: "))
    pos = search_dll(first, x)
    if pos > 0: print(f"Found at position {pos}")
    else:
      print("Not found")
  elif ch == 4:
    x = int(input("Enter data to insert: "))
    p = int(input("Enter position to insert: "))
    first = insert_dll(first, x, p)
  elif ch == 5:
    p = int(input("Enter position to delete: "))
    first = delete_dll(first, p)
  elif ch == 6:
    print("Exiting...")
    break
  else:
    print("Invalid choice")
__name__ == "__main__":
main()
```

CASE STUDY

Implementation of Stack and Queue Operations Using Linked lists.

Problem Statement

In traditional implementations, Stack and Queue are often built using arrays. However, arrays have fixed size limits and can result in wasted memory or overflow errors. To overcome these limitations, linked lists can be used to implement dynamically resizable stacks and queues.

Real-World Context

- **Stack:** Used in function call management, undo operations, expression evaluation.
- **Queue:** Used in task scheduling, print queues, and real-time systems like ticket booking and OS process scheduling.

Using linked lists ensures that memory is allocated only when needed, allowing these data structures to grow or shrink efficiently at runtime.

Data Structures Used

- **Singly Linked List**
  Node contains data and next pointer.
  - ➢ **Stack (LIFO):** Insert and remove from head.
  - ➢ **Queue (FIFO):** Insert at tail, remove from head.

Advantages of Linked List-Based Implementation

Feature	Array	Linked List
Dynamic size	No✗	Yes✓
Memory allocation	Pre-allocated	On demand
Overflow possibility	Yes	No (until memory full)
Insert/Delete efficiency	O(n) worst case	O(1) at head/tail

Stack using LL:

```
class Node:
  def __init__(self, data):
    self.data = data
    self.next = None
class Stack:
  def __init__(self):
    self.top = None
  def push(self, val):
    new_node = Node(val)
    new_node.next = self.top
    self.top = new_node
  def pop(self):
    if self.top is None:
      print("Stack Underflow")
      return
    temp = self.top
    self.top = self.top.next
    del temp
  def peek(self):
    return self.top.data if self.top else -1
  def is_empty(self):
    return self.top is None
  def display(self):
    current = self.top
    while current:
      print(f"{current.data} -> ", end="")
      current = current.next
    print("NULL")
# Usage
if __name__ == "__main__":
  s = Stack()
  s.push(10)
  s.push(20)
  s.push(30)
  s.display()  # 30 -> 20 -> 10 -> NULL
  s.pop()
  s.display()  # 20 -> 10 -> NULL
  print("Top:", s.peek())  # Top: 20
```

Queue using Linked List

```
class Node:
  def __init__(self, data):
    self.data = data
    self.next = None
class Queue:
  def __init__(self):
    self.front = None
```

```
        self.rear = None
    def enqueue(self, val):
        new_node = Node(val)
        if self.rear is None:
            self.front = self.rear = new_node
            return
        self.rear.next = new_node
        self.rear = new_node
    def dequeue(self):
        if self.front is None:
            print("Queue Underflow")
            return
        temp = self.front
        self.front = self.front.next
        if self.front is None:
            self.rear = None
        del temp
    def peek(self):
        return self.front.data if self.front else -1
    def is_empty(self):
        return self.front is None
    def display(self):
        current = self.front
        while current:
            print(f"{current.data} <- ", end="")
            current = current.next
        print("NULL")
# Usage
if __name__ == "__main__":
    q = Queue()
    q.enqueue(100)
    q.enqueue(200)
    q.enqueue(300)
    q.display()  # 100 <- 200 <- 300 <- NULL
    q.dequeue()
    q.display()  # 200 <- 300 <- NULL
    print("Front:", q.peek())  # Front: 200
```

Complexity Analysis

Operation	Stack (Linked List)	Queue (Linked List)
Push/Enqueue	O(1)	O(1)
Pop/Dequeue	O(1)	O(1)
Peek	O(1)	O(1)
Display	O(n)	O(n)
Space (n elements)	O(n)	O(n)

Enhancements

- Use doubly linked list if two-way traversal or deletion from both ends is needed.
- Add size counter for quick size retrieval.
- Add exception handling for production-grade code.

Takeaways

- Linked lists offer dynamic memory management.
- Ideal when the maximum size of the data structure is not known in advance.
- Stack and Queue using linked list support fast O(1) operations with efficient memory use.

EXERCISE

1. Explain basic stack operations (PUSH, POP, PEEK, and isEmpty). Write a Python function to implement PUSH and POP in a stack using arrays (lists).
2. What are multiple stacks? Give an example scenario where implementing multiple stacks in a single list (array) is useful.
3. Explain how stacks can be used for expression conversion. Write the algorithm to convert an infix expression to postfix using Python.
4. Write a step-by-step evaluation of the postfix expression: 5 6 2 + * 12 4 / -. Show stack status after each step.
5. List all queue operations. Implement the enqueue and dequeue functions using an array-based linear queue in Python.
6. Define Circular Queue. How is it different from a linear queue? Write a Python function to perform the enqueue operation in a circular queue.
7. What is a Priority Queue? Explain its advantages over a regular queue. List two real-world applications.
8. What are the different types of linked lists? Compare singly, circular, and doubly linked lists in terms of structure and use cases.
9. Describe and implement any three primitive operations (e.g., Insert, Delete, Traverse) on a singly linked list using Python.
10. Write a short program or pseudocode to concatenate two singly linked lists in Python. Explain how the next pointer is updated during this process.

✠ ✠ ✠